DATABASE MANAGEMENT SYSTEM LAB

ASSIGNMENT NO.7

Name: Nishtha Pramod Parve

PRN : 124B1F003

Case Study: Airline Management System

Mini Project: Centralized Club Management System

Case Study: Airline Management System

Aim: No SQL: Design and Develop Queries using CRUD operations. (Use CRUD operations, SAVE method, logical operators, joins etc.).

MongoDB is a NoSQL database that stores data in **JSON-like documents (BSON)** instead of tables.

Features of MongoDB

- Schema-less (flexible structure)
- Stores data as documents
- High performance and scalability

CRUD Operations

- **Create** → insertOne(), insertMany()
- **Read** → find()
- **Update** → updateOne(), updateMany()
- **Delete** → deleteOne(), deleteMany()

1. CREATE (Insert / Save Data)

Insert One Passenger

```
db.passengers.insertOne({
  passenger_id: 501,
  name: "Rahul Mehta",
  age: 35,
  nationality: "Indian"
});
```

```
test> use airline
switched to db airline
airline> db.passengers.insertOne({
|   passenger_id: 501,
|   name: "Rahul Mehta",
|   age: 35,
|   nationality: "Indian"
| });
{
  acknowledged: true,
  insertedId: ObjectId('69dfbfc967be5a99b53682d1')
}
```

Insert Multiple Flights

```
db.flights.insertMany([
  {
    flight_id: 101,
    source: "Mumbai",
    destination: "Lucknow",
    status: "Scheduled"
  },
  {
    flight_id: 102,
    source: "Kolkata",
    destination: "Nagpur",
    status: "Scheduled"
  }
]);
```

```
airline> db.flights.insertMany([
|    {
|      flight_id: 101,
|      source: "Mumbai",
|      destination: "Lucknow",
|      status: "Scheduled"
|    },
|    {
|      flight_id: 102,
|      source: "Kolkata",
|      destination: "Nagpur",
|      status: "Scheduled"
|    }
| ]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69dfbff067be5a99b53682d2'),
    '1': ObjectId('69dfbff067be5a99b53682d3')
  }
}
```

Insert Booking (SAVE concept)

```
db.bookings.insertOne({
  booking_id: 1001,
  passenger_id: 501,
  flight_id: 101,
  seat_no: "12A",
  travel_class: "Economy"
});
```

```
airline> db.bookings.insertOne({
|   booking_id: 1001,
|   passenger_id: 501,
|   flight_id: 101,
|   seat_no: "12A",
|   travel_class: "Economy"
| });
{
  acknowledged: true,
  insertedId: ObjectId('69dfbffd67be5a99b53682d4')
}
```

2. READ (Find Data)

Find All Passengers

```
db.passengers.find();
```

```
airline> db.passengers.find();
[
  {
    _id: ObjectId('69dfbfc967be5a99b53682d1'),
    passenger_id: 501,
    name: 'Rahul Mehta',
    age: 35,
    nationality: 'Indian'
  }
]
```

Find with Condition (Logical AND)

```
db.passengers.find({
  age: { $gt: 25 },
  nationality: "Indian"
```

});

```
airline> db.passengers.find({
|    age: { $gt: 25 },
|    nationality: "Indian"
| });
[
  {
    _id: ObjectId('69dfbfc967be5a99b53682d1'),
    passenger_id: 501,
    name: 'Rahul Mehta',
    age: 35,
    nationality: 'Indian'
  }
]
airline> db.flights.find({
|    $or: [
|      { source: "Mumbai" },
|      { destination: "Chennai" }
|    ]
| });
[
  {
    _id: ObjectId('69dfbff067be5a99b53682d2'),
    flight_id: 101,
    source: 'Mumbai',
    destination: 'Lucknow',
    status: 'Scheduled'
  }
]
```

Using OR

db.flights.find({

  $or: [

    { source: "Mumbai" },

    { destination: "Chennai" }

  ]

});

```
airline> db.flights.find({
|    $or: [
|      { source: "Mumbai" },
|      { destination: "Chennai" }
|    ]
| });
[
  {
    _id: ObjectId('69dfbff067be5a99b53682d2'),
    flight_id: 101,
    source: 'Mumbai',
    destination: 'Lucknow',
    status: 'Scheduled'
  }
]
```

Using NOT

db.bookings.find({

  travel_class: { $ne: "Business" }

});

```
airline> db.bookings.find({
|    travel_class: { $ne: "Business" }
| });
[
  {
    _id: ObjectId('69dfbffd67be5a99b53682d4'),
    booking_id: 1001,
    passenger_id: 501,
    flight_id: 101,
    seat_no: '12A',
    travel_class: 'Economy'
  }
]
```

3. UPDATE (Modify / Save Changes)

Update One Document

db.passengers.updateOne(

  { passenger_id: 501 },

  { $set: { age: 36 } }

);

```
airline> db.passengers.updateOne(
|    { passenger_id: 501 },
|    { $set: { age: 36 } }
| );
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Update Many

db.flights.updateMany(

{ status: "Scheduled" },

{ $set: { status: "Delayed" } }

);

```
airline> db.flights.updateMany(
|    { status: "Scheduled" },
|    { $set: { status: "Delayed" } }
| );
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
```

4. DELETE (Remove Data)

Delete One

db.bookings.deleteOne({ booking_id: 1001 });

Delete Many

db.passengers.deleteMany({ age: { $lt: 18 } });

```
airline> db.bookings.deleteOne({ booking_id: 1001 });
{ acknowledged: true, deletedCount: 1 }
airline> db.passengers.deleteMany({ age: { $lt: 18 } });
{ acknowledged: true, deletedCount: 0 }
```

5. JOIN in MongoDB (Using $lookup)

MongoDB uses **aggregation pipeline** instead of joins

Passenger + Booking Join

```
db.bookings.aggregate([
  {
    $lookup: {
      from: "passengers",
      localField: "passenger_id",
      foreignField: "passenger_id",
      as: "passenger_details"
    }
  }
]);
```

```
airline> db.bookings.aggregate([
|    {
|      $lookup: {
|        from: "passengers",
|        localField: "passenger_id",
|        foreignField: "passenger_id",
|        as: "passenger_details"
|      }
|    }
| ]);
```

Booking + Flight Join

```
db.bookings.aggregate([
  {
    $lookup: {
      from: "flights",
      localField: "flight_id",
      foreignField: "flight_id",
      as: "flight_details"
    }
  }
]);
```

```
airline> db.bookings.aggregate([
|    {
|      $lookup: {
|        from: "flights",
|        localField: "flight_id",
|        foreignField: "flight_id",
|        as: "flight_details"
|      }
|    }
| ]);
```

Mini Project : Centralized Club Management System

CRUD Queries

1. CREATE (Insert / Save Data)

Insert Member

```
db.club_members.insertOne({
  m_id: 10,
  m_name: "Rohan",
  m_email: "rohan@gmail.com",
  domain: "AI",
  club_id: 1
});
```

Insert Event

```
db.event.insertOne({
  event_id: 10,
  event_name: "Tech Talk",
  club_id: 1,
  start_date: "2026-05-01"
});
```

Insert Attendance (SAVE concept)

```
db.attendance.insertOne({
  attendance_id: 10,
  member_id: 10,
  event_id: 1,
  status: "Present"
});
```

```
test> use club
switched to db club
club> db.club_members.insertOne({
|   m_id: 10,
|   m_name: "Rohan",
|   m_email: "rohan@gmail.com",
|   domain: "AI",
|   club_id: 1
| });
{
  acknowledged: true,
  insertedId: ObjectId('69dfc98f139839ef423682d1')
}
club> db.event.insertOne({
|   event_id: 10,
|   event_name: "Tech Talk",
|   club_id: 1,
|   start_date: "2026-05-01"
| });
{
  acknowledged: true,
  insertedId: ObjectId('69dfc995139839ef423682d2')
}
club> db.attendance.insertOne({
|   attendance_id: 10,
|   member_id: 10,
|   event_id: 1,
|   status: "Present"
| });
{
  acknowledged: true,
  insertedId: ObjectId('69dfc99f139839ef423682d3')
}
```

2. READ (Find Data)

View All Members

```
db.club_members.find();
```

Logical Operator (AND)

```
db.club_members.find({
  club_id: 1,
  domain: "AI"
});
```

Logical Operator (OR)

```
db.event.find({
  $or: [
    { event_name: "Hackathon" },
    { event_name: "Dance Fest" }
  ]
```

});

Logical Operator (NOT)

```
db.attendance.find({
  status: { $ne: "Absent" }
});
```

```
club> db.club_members.find();
[
  {
    _id: ObjectId('69dfc98f139839ef423682d1'),
    m_id: 10,
    m_name: 'Rohan',
    m_email: 'rohan@gmail.com',
    domain: 'AI',
    club_id: 1
  }
]
club> db.club_members.find({
|   club_id: 1,
|   domain: "AI"
| });
[
  {
    _id: ObjectId('69dfc98f139839ef423682d1'),
    m_id: 10,
    m_name: 'Rohan',
    m_email: 'rohan@gmail.com',
    domain: 'AI',
    club_id: 1
  }
]
club> db.event.find({
|   $or: [
|     { event_name: "Hackathon" },
|     { event_name: "Dance Fest" }
|   ]
| });

club> db.attendance.find({
|   status: { $ne: "Absent" }
| });
[
  {
    _id: ObjectId('69dfc99f139839ef423682d3'),
    attendance_id: 10,
    member_id: 10,
    event_id: 1,
    status: 'Present'
  }
]
```

3. UPDATE (Modify / Save Changes)

Update Member Domain

```
db.club_members.updateOne(
  { m_id: 10 },
  { $set: { domain: "Machine Learning" } }
);
```

Update Many Records

```
db.attendance.updateMany(
  { status: "Absent" },
  { $set: { status: "Present" } }
);
```

```
club> db.club_members.updateOne(
|   { m_id: 10 },
|   { $set: { domain: "Machine Learning" } }
| );
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
club> db.attendance.updateMany(
|   { status: "Absent" },
|   { $set: { status: "Present" } }
| );
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 0
}
```

4. DELETE (Remove Data)

Delete One Member

```
db.club_members.deleteOne({ m_id: 10 });
```

Delete Multiple Records

```
db.attendance.deleteMany({ status: "Absent" });
```

```
club> db.club_members.deleteOne({ m_id: 10 });
{ acknowledged: true, deletedCount: 1 }
club> db.attendance.deleteMany({ status: "Absent" });
{ acknowledged: true, deletedCount: 0 }
```

5. JOIN (Using $lookup)

MongoDB uses **aggregation instead of joins**

Member + Club Join

```
db.club_members.aggregate([
 {
  $lookup: {
   from: "club",
   localField: "club_id",
   foreignField: "club_id",
   as: "club_details"
  }
 }
]);
```

Attendance + Member Join

```
db.attendance.aggregate([
 {
  $lookup: {
   from: "club_members",
   localField: "member_id",
   foreignField: "m_id",
   as: "member_details"
  }
 }
]);
```

```
club> db.club_members.aggregate([
|     {
|       $lookup: {
|         from: "club",
|         localField: "club_id",
|         foreignField: "club_id",
|         as: "club_details"
|       }
|     }
| ]);

club> db.attendance.aggregate([
|     {
|       $lookup: {
|         from: "club_members",
|         localField: "member_id",
|         foreignField: "m_id",
|         as: "member_details"
|       }
|     }
| ]);
[
  {
    _id: ObjectId('69dfc99f139839ef423682d3'),
    attendance_id: 10,
    member_id: 10,
    event_id: 1,
    status: 'Present',
    member_details: []
  }
]
```

Conclusion

MongoDB provides a flexible, document-based approach for managing data in both Airline Management System and Centralized Club Management System. Unlike traditional relational databases, MongoDB allows data to be stored in JSON-like documents, making it easier to handle dynamic and large-scale data efficiently.

In the Airline Management System, MongoDB helps in managing passenger details, flight information, and booking records, enabling quick updates and retrieval of data. Similarly, in the Centralized Club Management System, it is used to manage club members, events, attendance, and budget requests in a structured yet flexible manner.

CRUD operations (Create, Read, Update, Delete) form the core of both systems, allowing efficient data manipulation. The concept of saving data using insert and update operations ensures proper data storage and modification. Logical operators such as AND, OR, and NOT help in filtering and retrieving relevant data based on conditions.

Additionally, MongoDB's aggregation framework and $lookup operation allow combining and analyzing data across collections, similar to joins in SQL. Indexing further improves performance by enabling faster data retrieval.

Overall, MongoDB proves to be highly suitable for both airline and centralized club management systems due to its scalability, flexibility, and ability to efficiently handle complex and large datasets in real-world applications.